Fpt University

TRƯỜNG ĐẠI HỌC FPT

CAPSTONE PROJECT REPORT

Report 4 – Software Design Document

Ho Chi Minh City, 31 October 2026

Table of Contents

I. Record of Changes....3
II. Software Design Document....3
1. System Design.... 3
1.1 System Architecture....3
1.1.1 Authority and Data Boundaries....5
1.1.2 Main Interface Contracts....5
1.1.3 Deployment View (Logical)....5
1.1.4 Technology, DevOps and External Services....6
1.2 Package Diagram....6
2. Database Design....7
2.1 Data Storage Strategy.... 7
2.2 Relational Data Model (PostgreSQL).... 7
2.2.1 Table Descriptions – Accounts and Economy....8
2.2.2 Table Descriptions – Match, Reward and AI.... 9
2.2.3 Persistence Integrity Rules.... 9
2.3 Telemetry Document Model (MongoDB).... 9
3. Detailed Design....10
3.1 Account Authentication and Profile.... 10
3.2 Multiplayer Lobby and Session Lifecycle.... 11
3.3 Player Interaction, Life State and Revive....14
3.4 Expedition Mission and Objective Control.... 16
3.5 Stalker Monster AI.... 18
3.6 Adaptive Experience Director and Telemetry....20
3.7 Match Result and Reward Processing....21
3.8 Virtual Shop, Wallet and Inventory.... 22
3.9 UI, HUD and Gameplay Feedback....24
3.10 Main Business Workflows....26

I. Record of Changes

Date	A/M/D	In charge	Change Description
28/10/2026	A	ECHO PROTOCOL team	Initial SDD using the school Report 4 outline, Report 1–3 and the inspected GitHub source. Player participation scope fixed at 1–4.
30/10/2026	M	Lâm Thanh Phúc	Selected LTP source; added versioned Legal Agreements lifecycle and consent entity, corrected Adaptive AI/fixed fallback boundary; registry unchanged.
31/10/2026	M	Nguyễn Nhật Lệ Thuyên	Corrected interaction class descriptions, aligned LTP technology and deployment references, and added the agreement acceptance ERD extension.
31/10/2026	A	ECHO PROTOCOL team	Added four main business workflows (WF-01–WF-04); WF-03 documents Stalker gameplay and player survival.

A = Added; M = Modified; D = Deleted.

II. Software Design Document

1. System Design

1.1 System Architecture

ECHO PROTOCOL separates real-time session state from persistent backend state. Unity owns player presentation and local input; Photon Fusion Host Mode provides network session connectivity, while the session Host validates critical gameplay changes. ASP.NET Core handles authenticated APIs, game results, rewards, wallet operations, inventory, and AI records. PostgreSQL is the transactional store, and MongoDB stores gameplay telemetry.

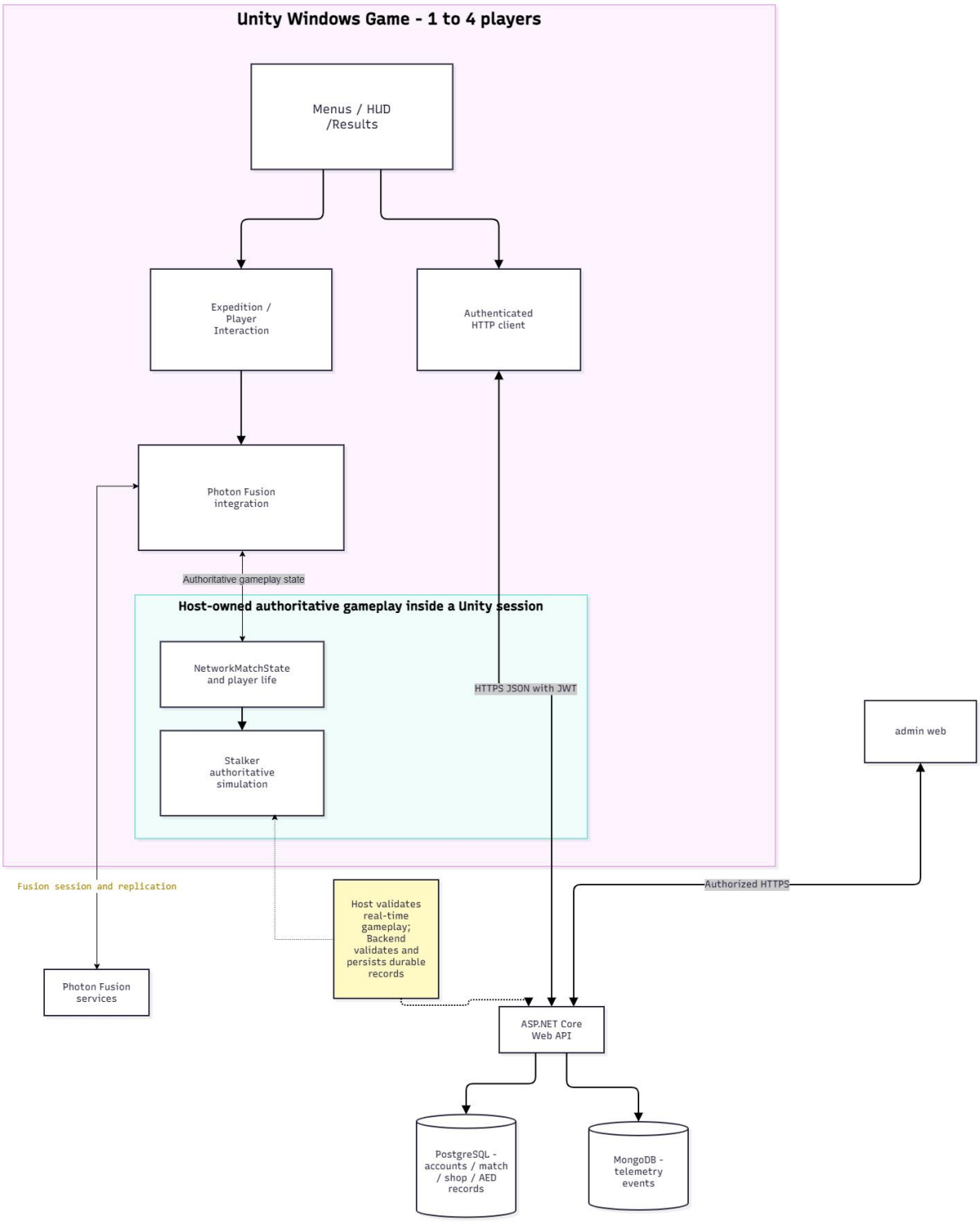


Figure 1. Overall logical architecture and external interfaces

Component	Key responsibility	Integration
Unity client	Input, player actions, expedition controllers, AI presentation, HUD, menu, result and shop	Photon networking; HTTPS API
Photon Fusion / Host	Session membership, network runner, replicated state; host-only authoritative gameplay mutation	Fusion protocol / callbacks / RPC
ASP.NET Core API	Authentication, authority binding, result validation, reward, shop, roles, scenario records	HTTPS JSON / JWT
PostgreSQL	Relational identity, progression, economy, match	EF Core / Npgsql

Component	Key responsibility	Integration
	and scenario records	
MongoDB	Backend-ingested gameplay event telemetry.	MongoDB driver / repository
Admin web	View limited protected user/economy data and permitted auditable changes	Authenticated Backend API

1.1.1 Authority and Data Boundaries

Player requests do not directly own final mission state. Locally collected input or interaction commands are validated at State Authority before any corresponding replicated state is changed. Client HUD and effects read the accepted Networked state. The Backend must independently validate Host identity and registered match before committing results, rewards, and wallet transactions; a client display is not evidence of server acceptance.

1.1.2 Main Interface Contracts

Interface	Transport / Authority	Design constraints
Unity <-> Photon Fusion	NetworkRunner, NetworkBehaviour, RPC, Networked state	Host validation; no direct arbitrary client commit of objective/result
Unity / authorized Host <-> Backend	HTTPS REST + JWT	Validate identity/roster/permissions; return explicit rejection
Backend <-> PostgreSQL	EF Core transactions	Consistency, unique keys, idempotency for reward/purchase
Backend <-> MongoDB	Telemetry repository	Match identity, event sequence, timestamps and schema version
Admin Web <-> Backend	Authorized HTTPS API	RBAC enforced server-side, not only hidden UI controls
Unity/Web <-> Agreement API	HTTPS JSON/JWT; GET documents/status, POST accept	Current bundle/hash match; all required flags; 403 AGREEMENT_REQUIRED for protected operations; no cash/payment obligation added.

1.1.3 Deployment View (Logical)

ECHO PROTOCOL runs as a Windows Unity client connected to Photon Fusion Host Mode and an ASP.NET Core backend deployed on AWS EC2. GitHub Actions supports build and release workflows, while Docker Compose manages backend, web and database services. PostgreSQL and MongoDB connection settings are supplied by environment configuration. The design snapshot corresponds to LTP commit 3c853fec.

1.1.4 Technology, DevOps and External Services

Technology, integration services and workflows for the reviewed LTP baseline are listed below.

CATEGORY	VERSION / SERVICE	PURPOSE / DATA / VERIFICATION
Game engine	Unity 6000.5.8f1; C#; Photon Fusion Host Mode	Windows game, network session membership, replicated gameplay; Photon service receives session/connection data.
Backend / web	.NET 8 / ASP.NET Core; Next.js 16.4.3	Authenticated REST APIs; limited Admin UI. JSON/JWT, authorization and explicit errors.
Databases	PostgreSQL (EF Core/Npgsql 8.0.11); MongoDB.Driver 3.11.0	Relational user, match, reward and economy records; MongoDB event telemetry. Server versions depend on deployment.
Deployment / CI	GitHub Actions; Docker Compose; AWS EC2	GitHub Actions pipelines build and check backend/Unity code; Docker Compose supports AWS EC2 deployment and operational scripts.
External services	Photon Fusion; AWS	Photon Fusion handles network sessions; AWS hosts backend/data services.
AI tools used	OpenAI ChatGPT (GPT-6)	Assisted with requirements, documentation, diagram/source review and test planning; project team verifies technical decisions.

1.2 Package Diagram

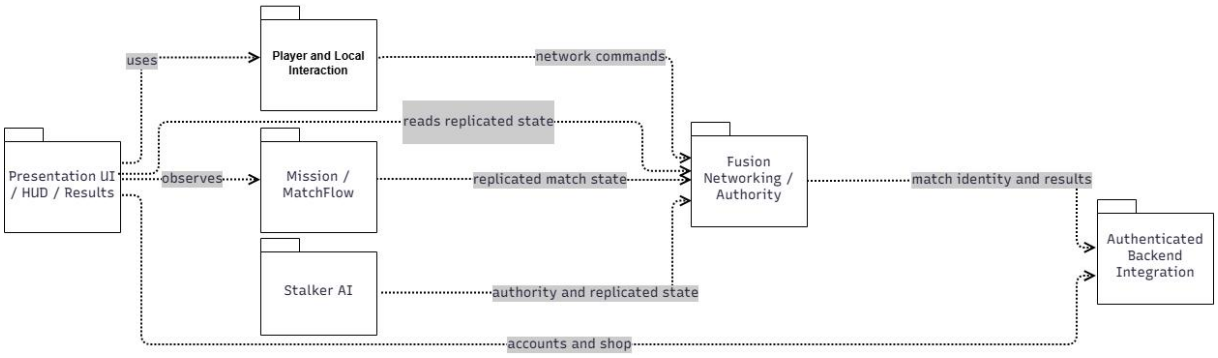


Figure 2. Unity gameplay and networking packages

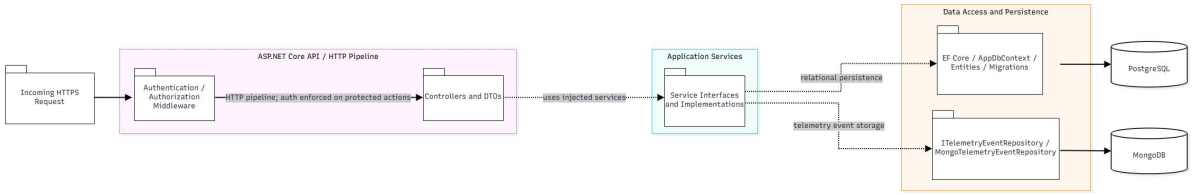


Figure 3. Backend and persistence packages

No.	Package	Description / source area
01	Presentation	MainMenu, Lobby UI, HUD, Result and shop presentation components.
02	Player & Interaction	Handles local player input, equipment and interaction prompts.
03	Mission & Match Flow	MatchFlowController, energy cores, objectives, power/security, evacuation.
04	Stalker AI	StalkerController, StalkerVisionSensor, StalkerFusionRuntime, scenario resolution.
05	Networking & Authority	Manages Fusion sessions, RPCs, player life state, replicated match state and Host authority.
06	Backend API / Services	Controllers, DTOs, authentication, match, reward, purchase, inventory and admin services.
07	Persistence & Telemetry	AppDbContext entities/migrations; MongoTelemetryEventRepository.

2. Database Design

2.1 Data Storage Strategy

PostgreSQL is the authoritative source for persistent user and transactional records. Tables are mapped with Entity Framework Core via AppDbContext. MongoDB stores event-oriented telemetry records used by analysis and adaptive inputs. Photon replicated state (e.g., current mission phase, player life and enemy state) is transient session state and should not be modeled as a database table unless it is explicitly persisted by a backend contract.

2.2 Relational Data Model (PostgreSQL)

The PostgreSQL logical model separates account/economy entities (Figure 4) from match, reward and adaptive-decision entities (Figure 5). A versioned acceptance record, UserAgreementAcceptances, references Users by UserId and uses the composite key (UserId, BundleVersion),

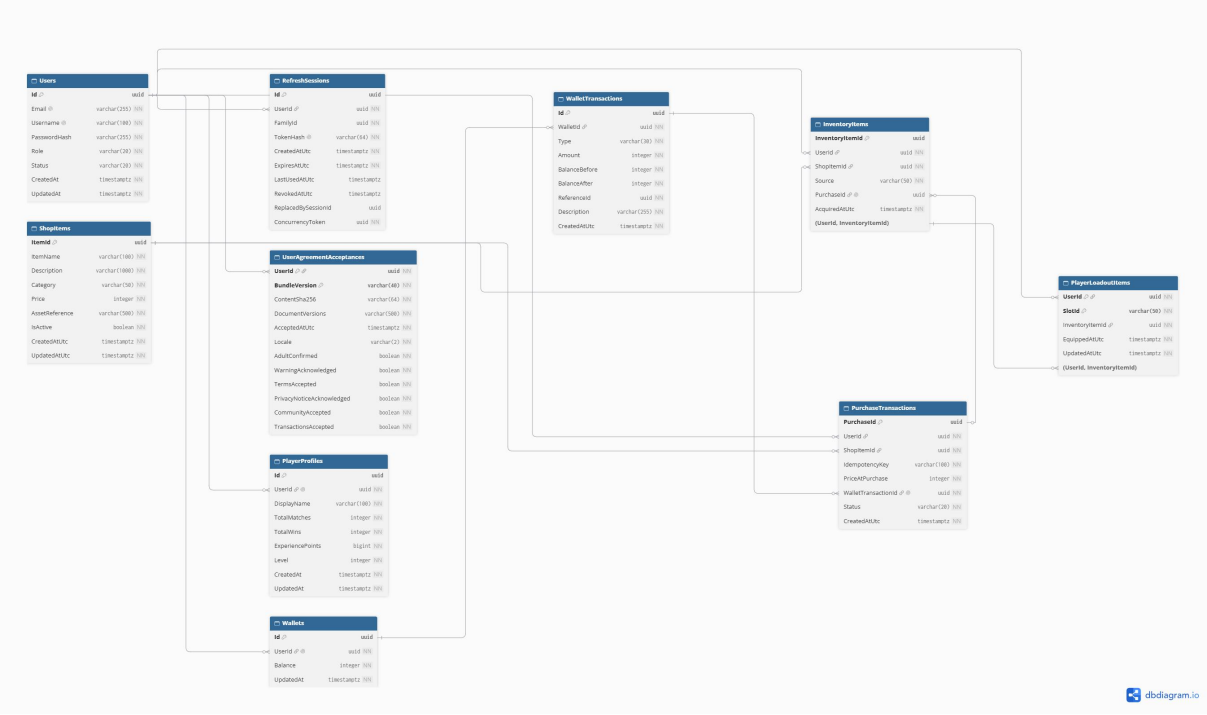


Figure 4. Core account and virtual economy ERD (logical cardinalities)

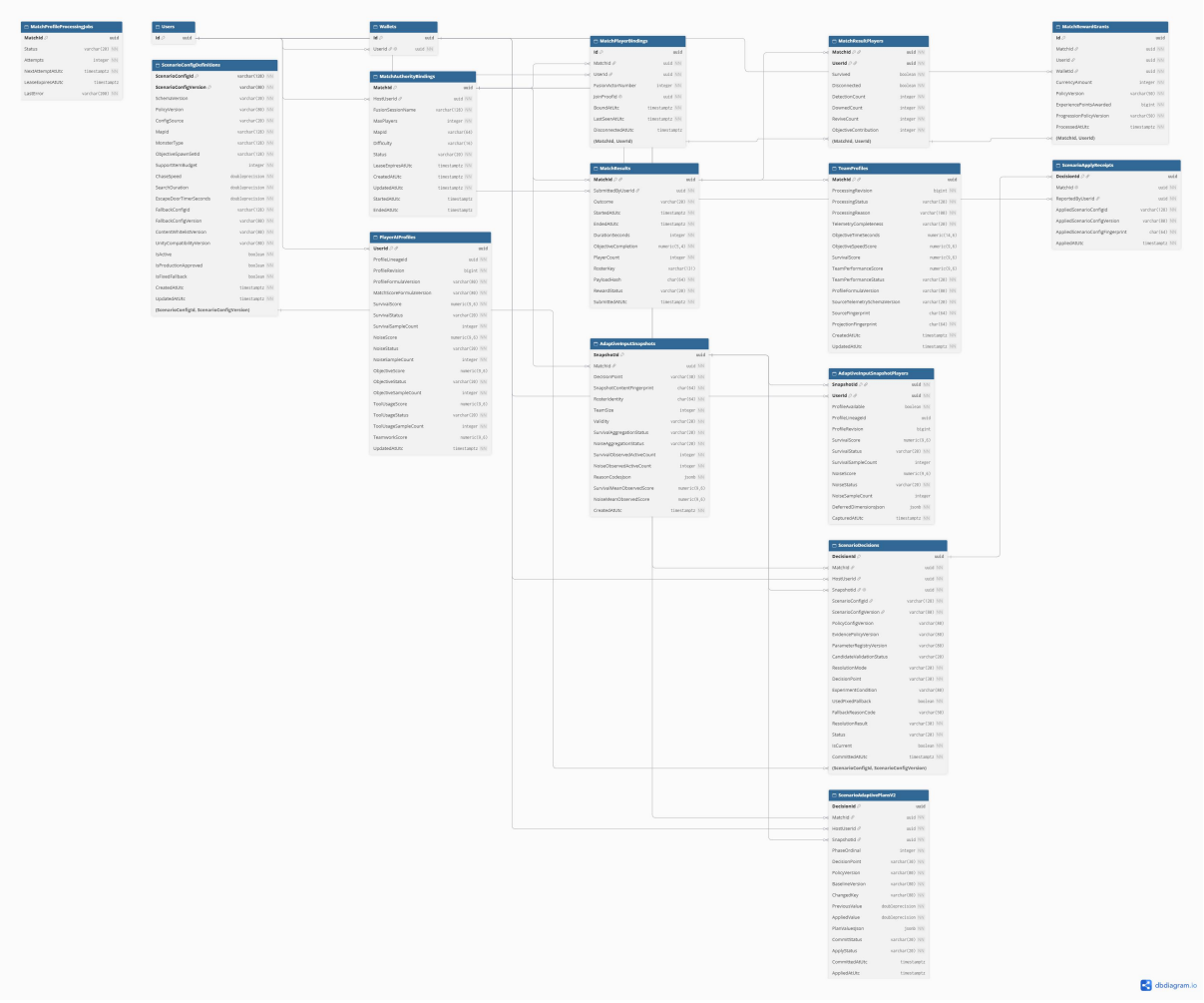


Figure 5. Core match, reward and adaptive-record relationships

2.2.1 Table Descriptions – Accounts and Economy

Table	Purpose / key fields
Users	PK Id; account identity, hashed password, role and status.
RefreshSessions	Authentication refresh sessions; user reference, token hash and expiry/concurrency constraints.
PlayerProfiles	Account progression and profile data; one-to-one user association.
Wallets	PK Id, FK UserId; persistent ECHO Credits balance.
WalletTransactions	PK Id, FK WalletId; type, amount, before/after balance, reference identity.
ShopItems	Eligible server-side catalogue, price, status and category.
PurchaseTransactions	PK PurchaseId, FK UserId, ShopItemId, WalletTransactionId; IdempotencyKey.
InventoryItems	PK InventoryItemId, FK UserId, ShopItemId, optional PurchaseId; ownership source.
PlayerLoadoutItems	Account equipment selections referencing owned compatible inventory. PK (UserId, SlotId); composite FK to owned inventory.
UserAgreementAcceptances	UserId FK → Users; composite PK (UserId, BundleVersion). Version/hash, document versions, UTC acceptance, locale and six acknowledgement flags; added by LTP migration 20261030090842.

2.2.2 Table Descriptions – Match, Reward and AI

Table	Purpose / key fields
MatchAuthorityBindings	PK MatchId; Host UserId, room binding, status, lease and selected match configuration.
MatchPlayerBindings	Participant identity/roster binding associated with authoritative match.
MatchResults	PK MatchId; authoritative match outcome, duration, submitted-by user and roster identity.
MatchResultPlayers	PK (MatchId, UserId); per-player match result and objective/survival statistics.
MatchRewardGrants	PK Id; MatchId, UserId, WalletId, CurrencyAmount, XP and policy versions.
MatchScores	Validated match scoring records associated with players/results.
PlayerAIProfiles	PK/FK UserId; aggregated player behaviour/profile data for AED inputs.
TeamProfiles	PK/FK MatchId; aggregated team-level behaviour/result data.
ScenarioConfigs / ScenarioContentDefinitions	Versioned allowed scenario configurations and content.
AdaptiveInputSnapshots / AdaptiveInputSnapshotPlayers	Pre-match observed input snapshot and constituent players. PK (SnapshotId, UserId); player snapshot data.
ScenarioDecisions / ScenarioApplyReceipts	Decision identity, version/fingerprint, fallback reason and application receipt. PK/FK DecisionId; records scenario application.

2.2.3 Persistence Integrity Rules

- Wallet balances must not be trusted from Unity input. PurchaseService checks eligible item, ownership and available balance, and creates ledger + purchase + inventory grant within a database transaction.
- A repeated purchase IdempotencyKey must not charge Credits again or duplicate ownership. Conflicting replay requests must be rejected.
- MatchResultService checks Host authorization, active match state, roster and accepted result identity before committing. RewardService grants the corresponding reward once, with replay handling.
- Physical constraints and indexes are defined in EF Core model configuration and migrations.

2.3 Telemetry Document Model (MongoDB)

Field	Type / role
Id	Guid; event document identity
MatchId	Guid; match correlation
UserId	Nullable Guid; user correlation if applicable
EventType	String; event classification

Field	Type / role
Ts	UTC timestamp of source event
EventSequence	Long; authoritative event ordering
ValueJson	BSON document; event payload
ReasonCode	Optional string; explanation for outcomes
SchemaVersion	String; compatibility/version
SemanticFingerprint	String; replay/content identity
IngestedAt	UTC ingestion timestamp

3. Detailed Design

3.1 Account Authentication and Profile

After authentication, Unity/Web requests agreement status and current documents through GET /api/agreements/status and GET /api/agreements/documents. The player accepts the matching bundle version/hash and required declarations via POST /api/agreements/accept. UserAgreementAcceptances stores the accepted version, locale, time and flags. AgreementGateMiddleware blocks protected match, scenario, write-inventory and purchase operations with 403 AGREEMENT_REQUIRED until consent is valid.

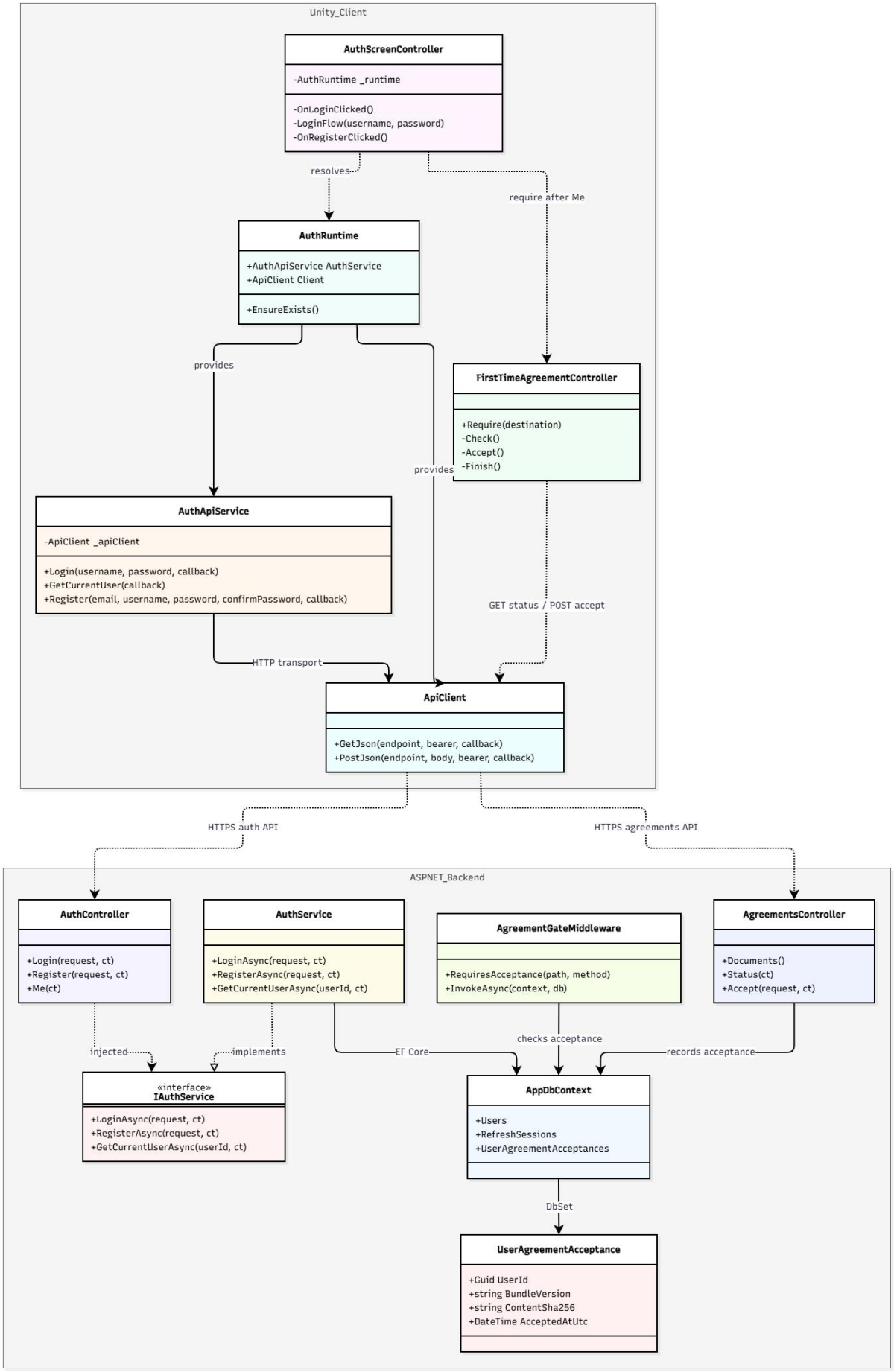


Figure 6. Selected authentication class/component diagram

Class / Component	Design responsibility / selected operations
AuthScreenController / AuthRuntime	Handles authentication UI and initializes Unity authentication services.
AuthApiService / ApiClient	Manages authentication requests, tokens and HTTP communication.
AuthController	Exposes protected authentication endpoints.
IAuthService / AuthService	Validates credentials and manages token issuance and refresh.
AppDbContext / User	Persists user identity, account status and authentication records.

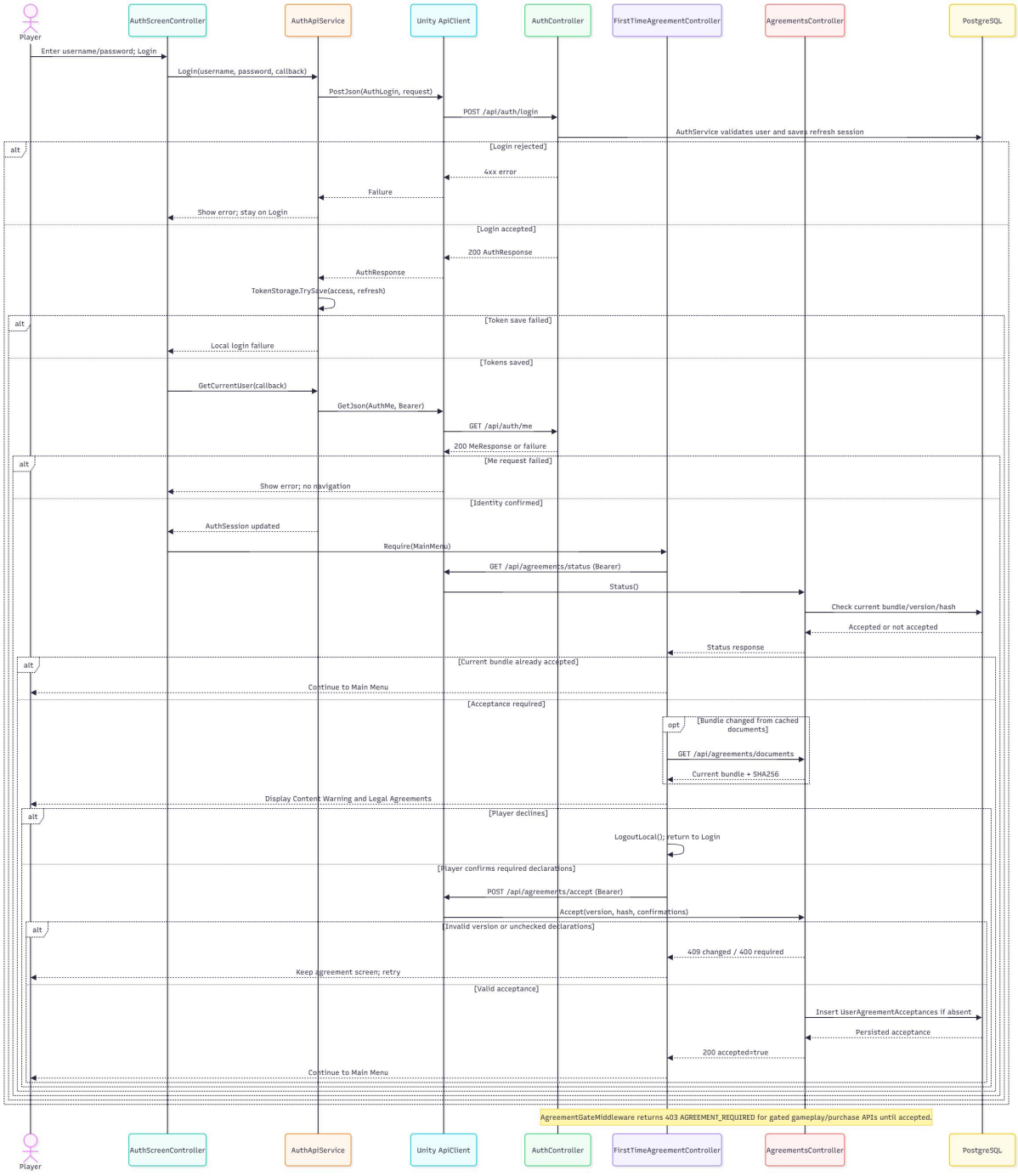


Figure 7. Login and authenticated session sequence

3.2 Multiplayer Lobby and Session Lifecycle

Field	Specification
Purpose	Create/join a private room, synchronize player readiness and start a host-authoritative Expedition.
Inputs / triggers	Room name, maximum room capacity, ready flags, selected difficulty/mode and Start input.
Authority / owner	NetworkBootstrap owns the runner; LobbyManager requests and validates Host-only start; MatchAuthorityRuntime binds backend authority.
Failure / negative path	Reject missing session, full/invalid room, non-host start, missing ready state or backend binding failure; never assume success on a failed transition.
SRS trace	FR-MUL-01–05; UC-03–05

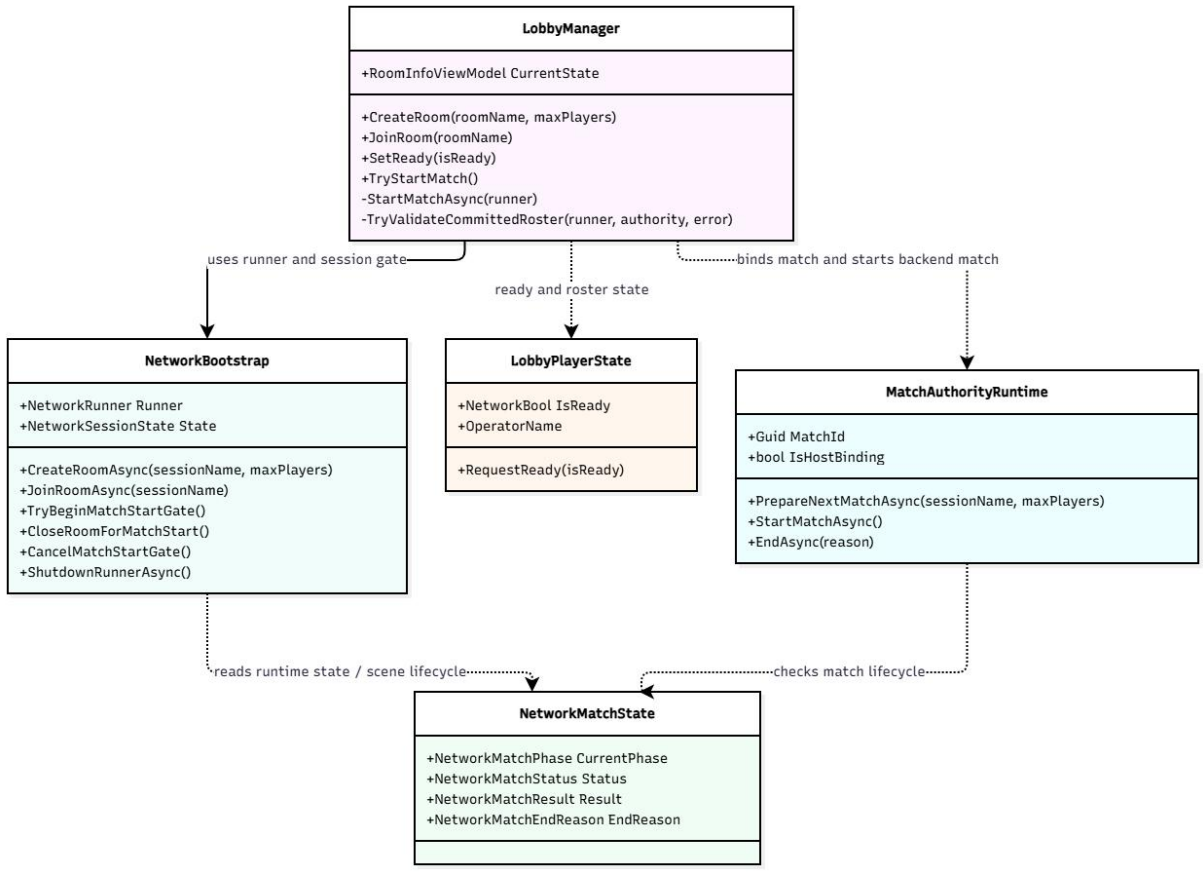


Figure 8. Lobby, runner and authoritative match state classes

Class / Component	Design responsibility / selected operations
NetworkBootstrap	NetworkRunner lifecycle; CreateRoomAsync, JoinRoomAsync, disconnect/reconnect and scene transitions.
LobbyManager / LobbyPlayerState	Lobby UI projection, ready and host-controlled match-start gate.
MatchAuthorityRuntime	Authoritative match binding and backend start flow.
NetworkMatchState	Replicated match state, phase and terminal outcome.

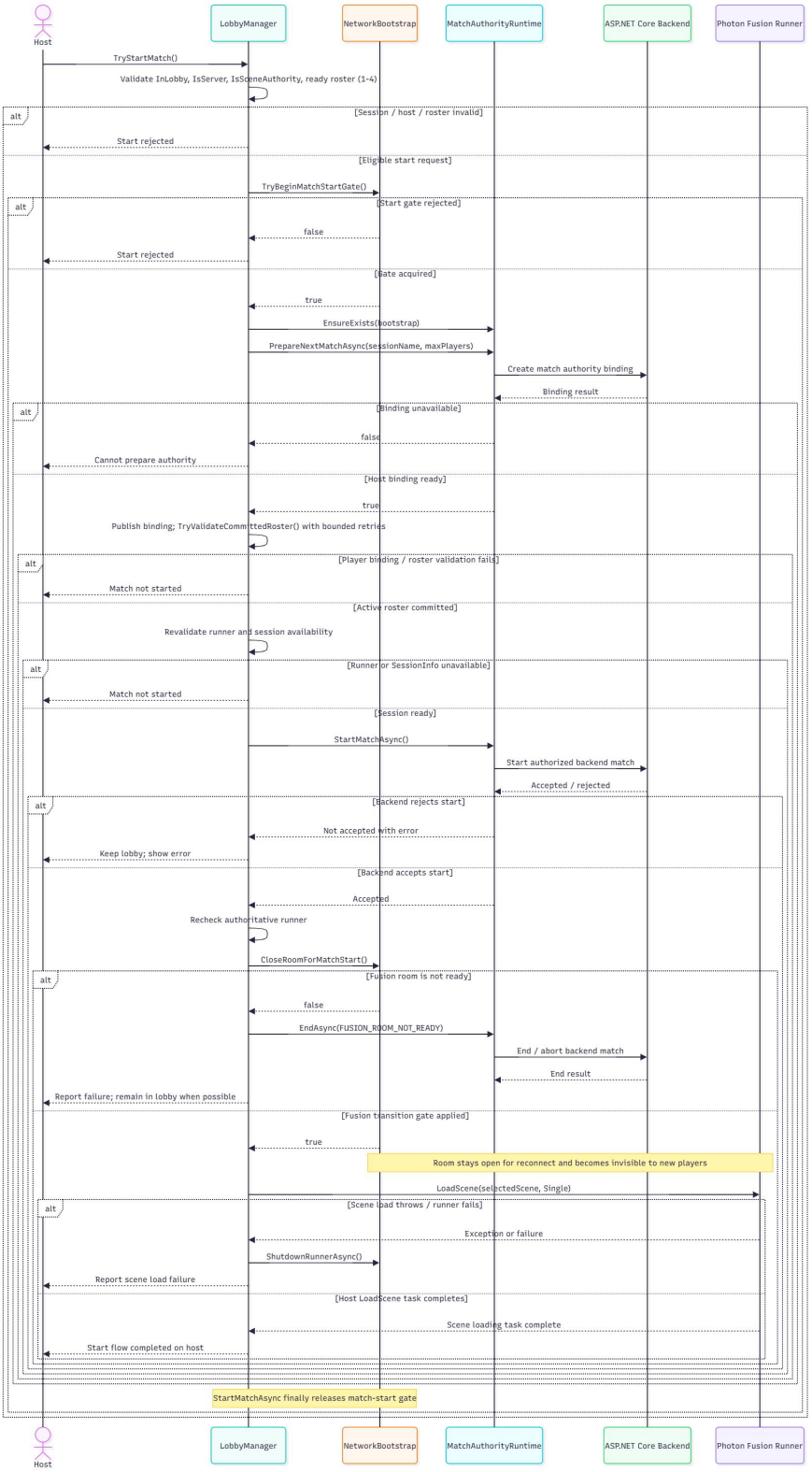


Figure 9. Host starts a private Expedition sequence (simplified)

Expedition supports 1–4 active players, including solo Host. The room capacity and number of active participants are separate; a Host-only start gate verifies membership/readiness and backend match authority before transitioning to gameplay.

3.3 Player Interaction, Life State and Revive

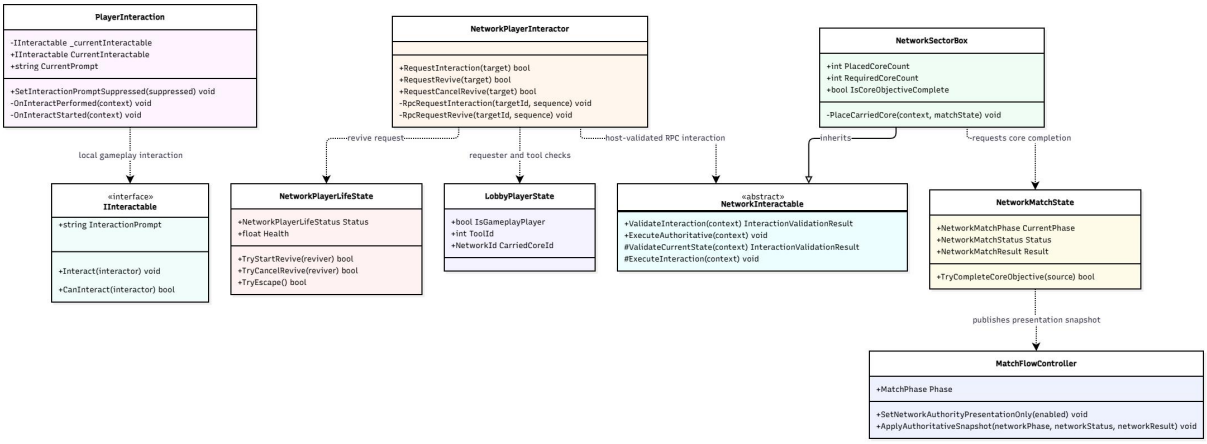


Figure 10. Selected player, objective and authoritative gameplay classes

Class / Component	Design responsibility / selected operations
PlayerInteraction / IInteractable	Detect eligible raycast target, issue interaction and publish prompt.
NetworkPlayerLifeState / Rules	Alive, Downed, Eliminated, Escaped and Caught; validate revive and damage.
NetworkMatchState	Game-wide authoritative state, ending and integration.
NetworkPlayerInteractor / NetworkInteractable	Sends and validates authoritative network interaction requests through Fusion RPCs.

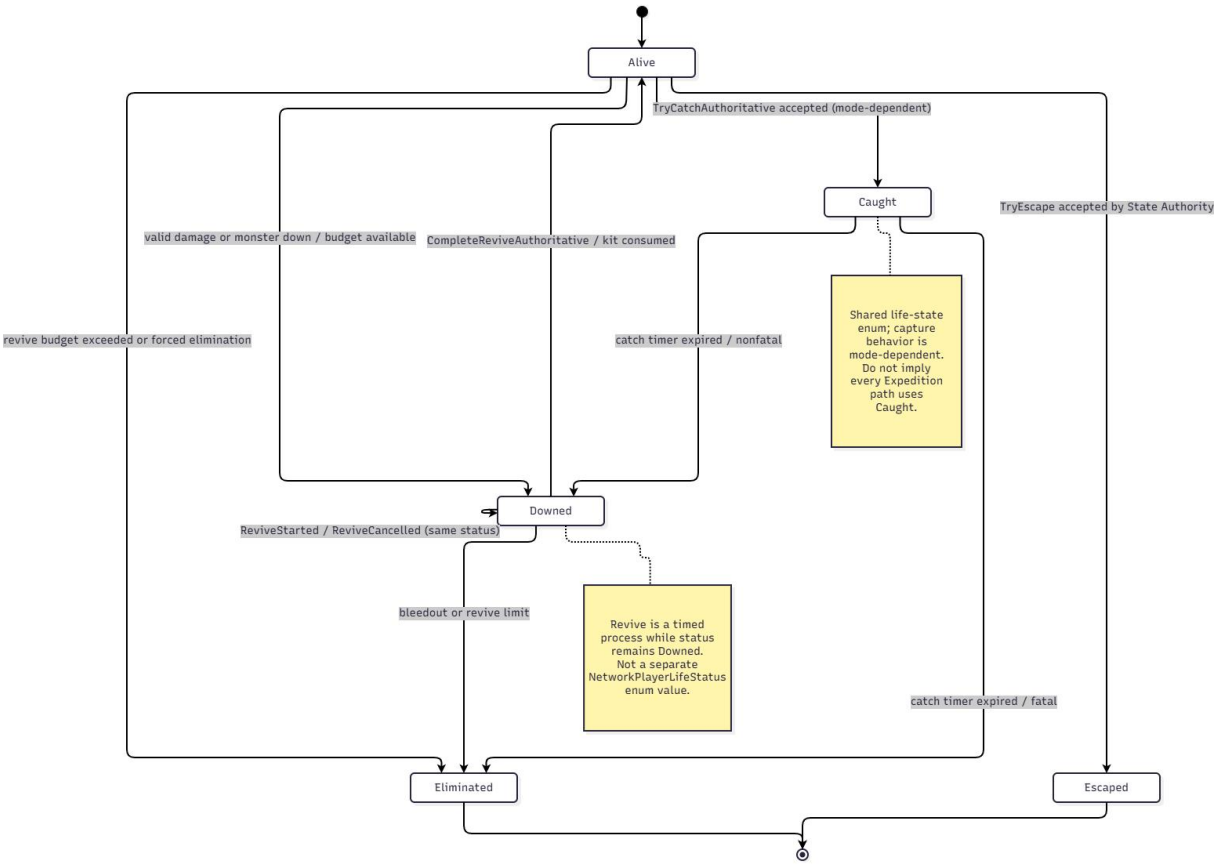


Figure 11. Player life-state transitions (abstracted from NetworkPlayerLifeStatus)

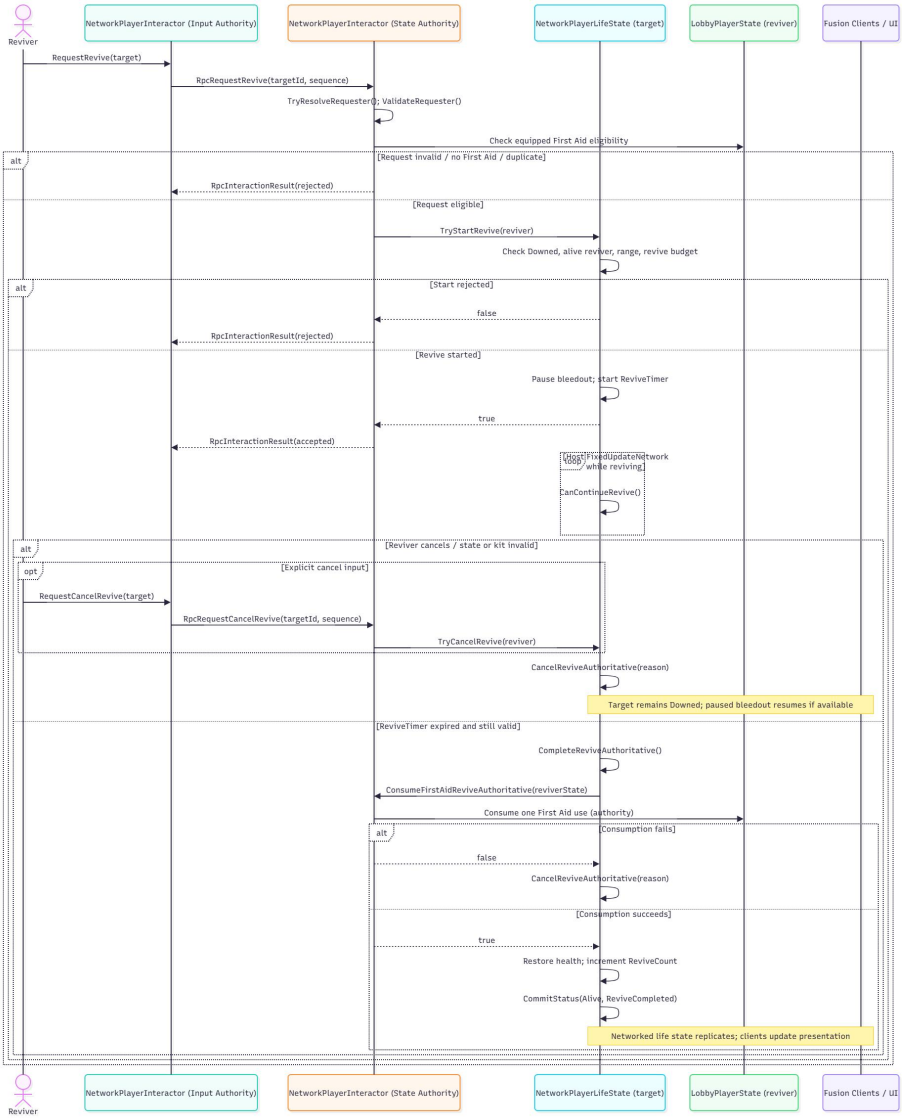


Figure 12. Eligible teammate revive sequence

3.4 Expedition Mission and Objective Control

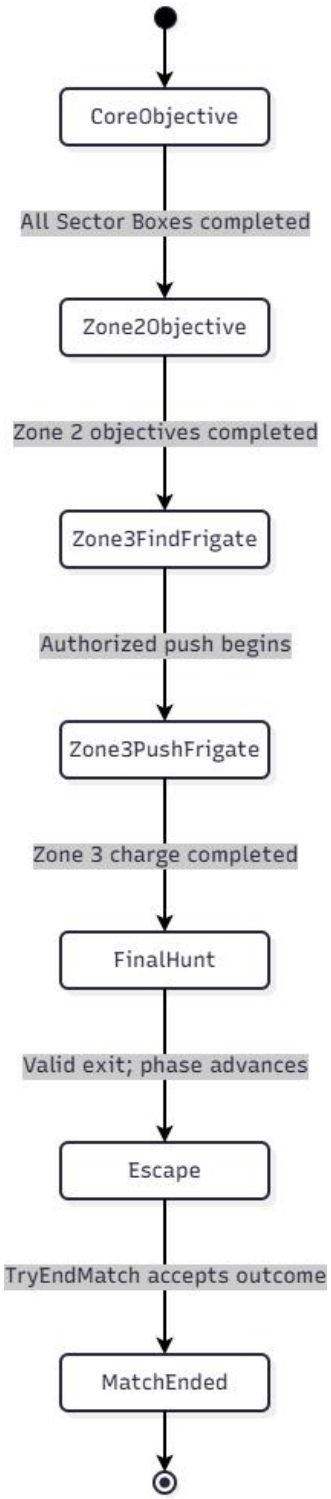


Figure 13. Primary Expedition phase progression (successful path)

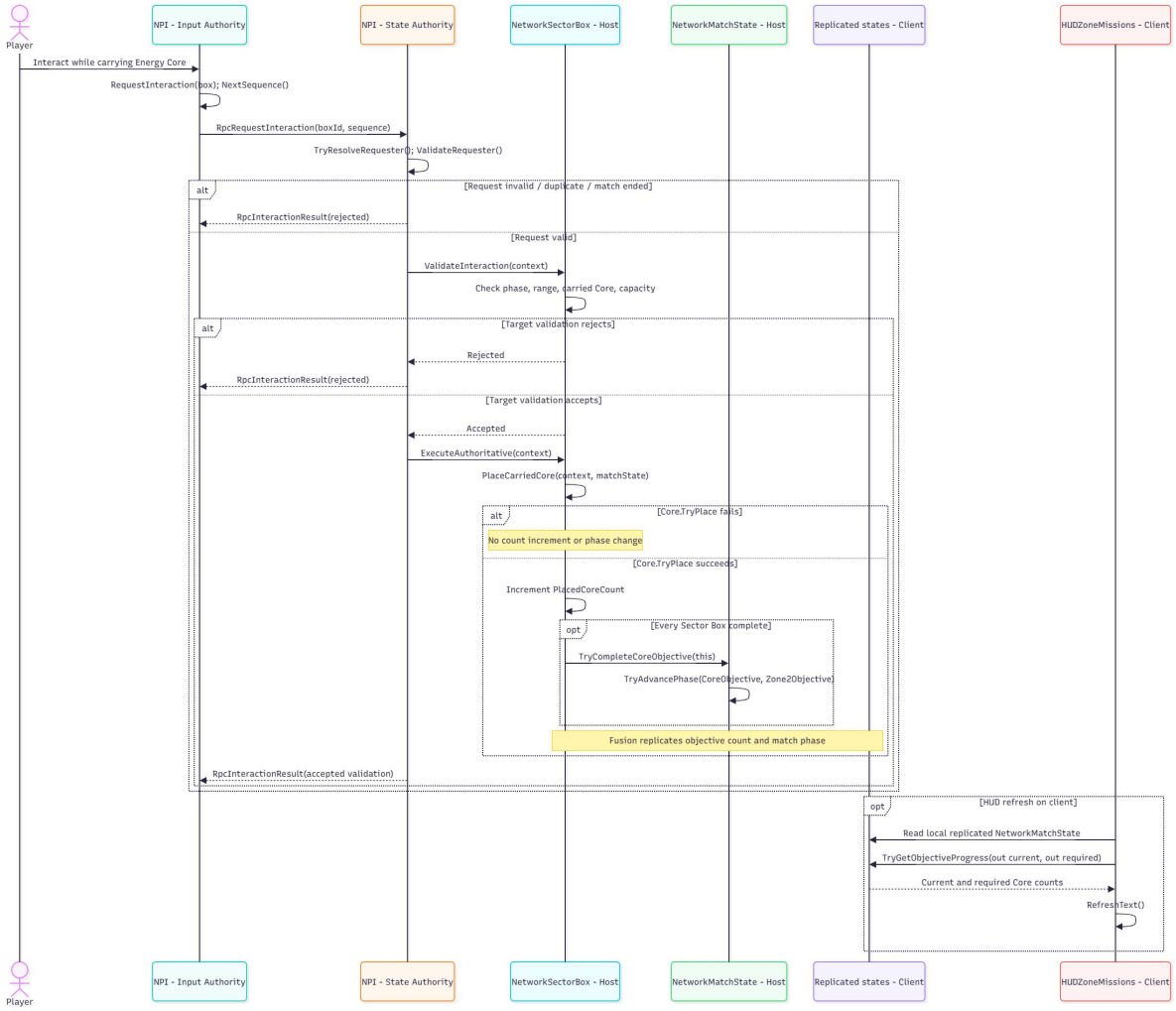


Figure 14. Authoritative objective interaction and HUD update sequence

Class / Component	Design responsibility / selected operations
NetworkPlayerInteractor	Routes interaction requests as RPCs to the owning State Authority, which validates them.
NetworkSectorBox	Validates Energy Core insertion against authoritative state and updates accepted objective progress.
NetworkMatchState	Owns replicated mission phase, objective flags and match result snapshots.
PlayerInteraction / IInteractable	Selects eligible local targets and displays interaction prompts; cannot directly commit shared state.
Zone2MissionDirector / Zone3MissionDirector	Manages mission prerequisites and scene presentation, using Host-accepted game state.

3.5 Stalker Monster AI

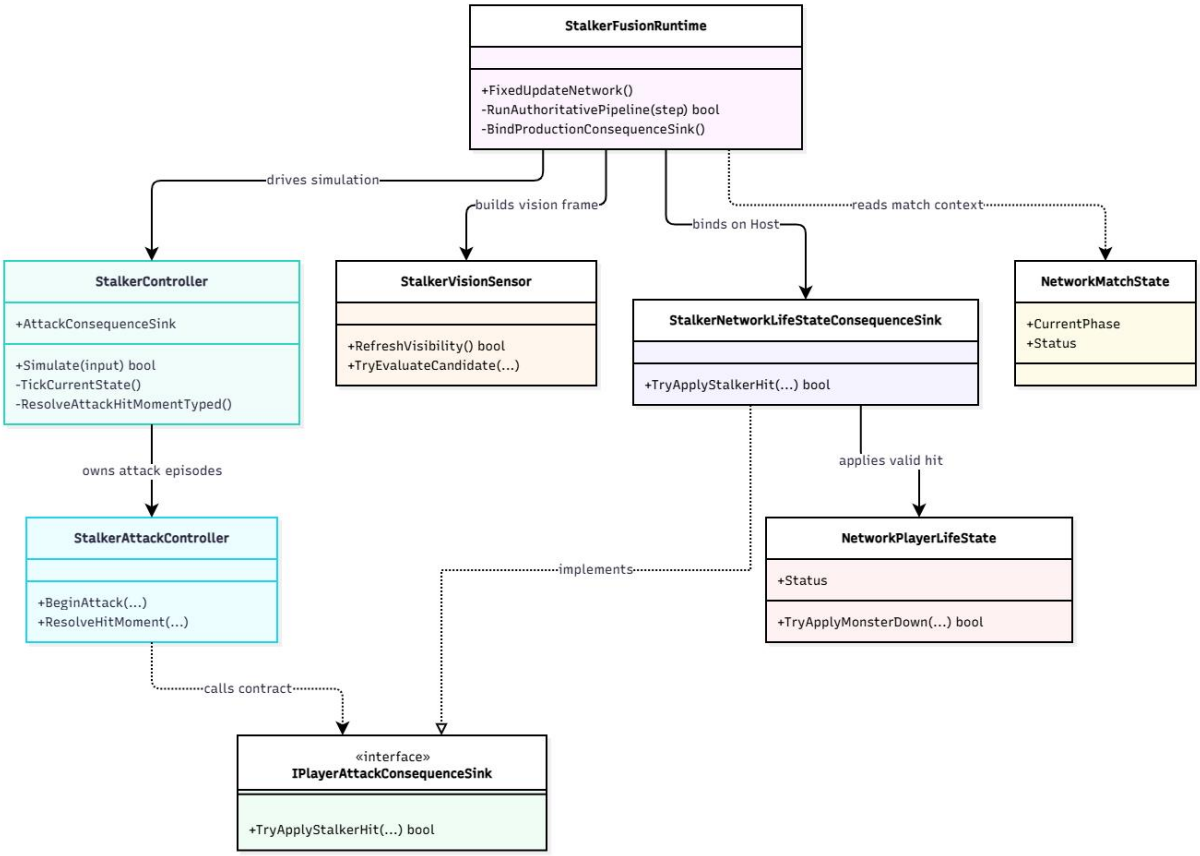


Figure 15. Stalker AI class/component diagram

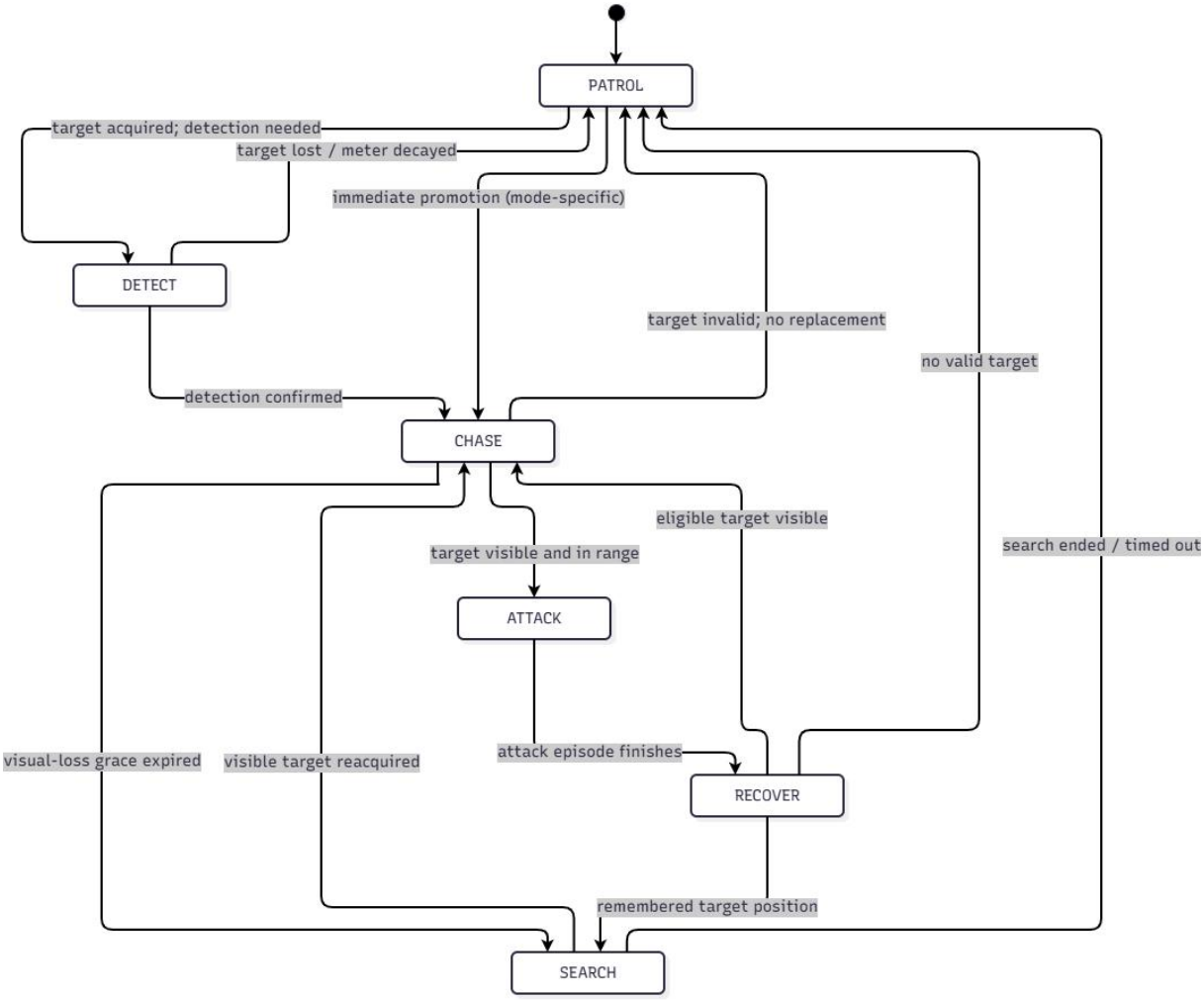


Figure 16. Primary Stalker finite-state machine (logical transitions)

StalkerState enum includes PATROL, DETECT, CHASE, ATTACK, RECOVER and SEARCH. The diagram expresses expected high-level transitions; it does not assert every runtime transition must traverse the same path. Exact vision angle, range, noise scoring, UV resource and cooldown are controlled by serialized/game settings and require scene tests.

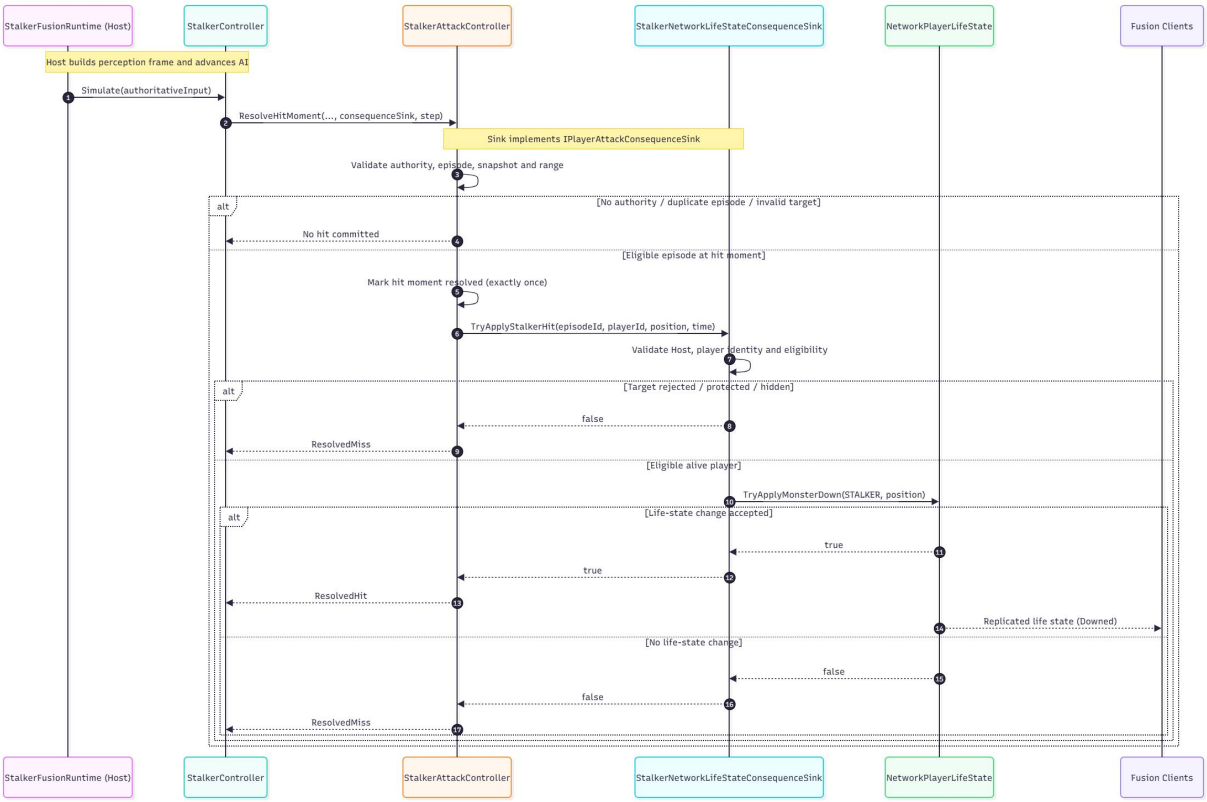


Figure 17. Host-authoritative Stalker attack and replicated outcome sequence

3.6 Adaptive Experience Director and Telemetry

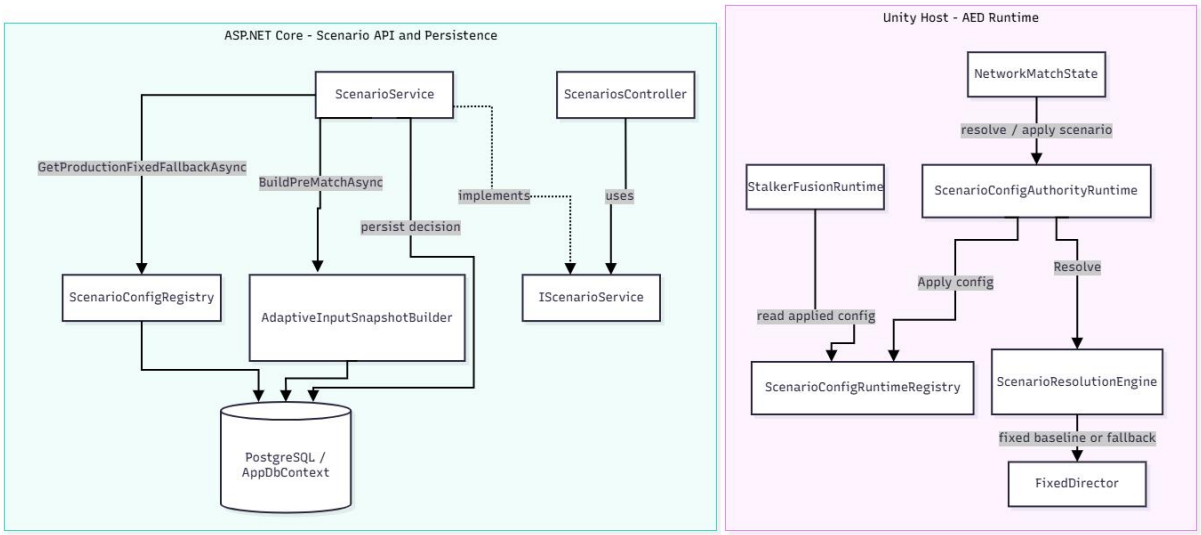


Figure 18. AED and pre-match scenario decision components

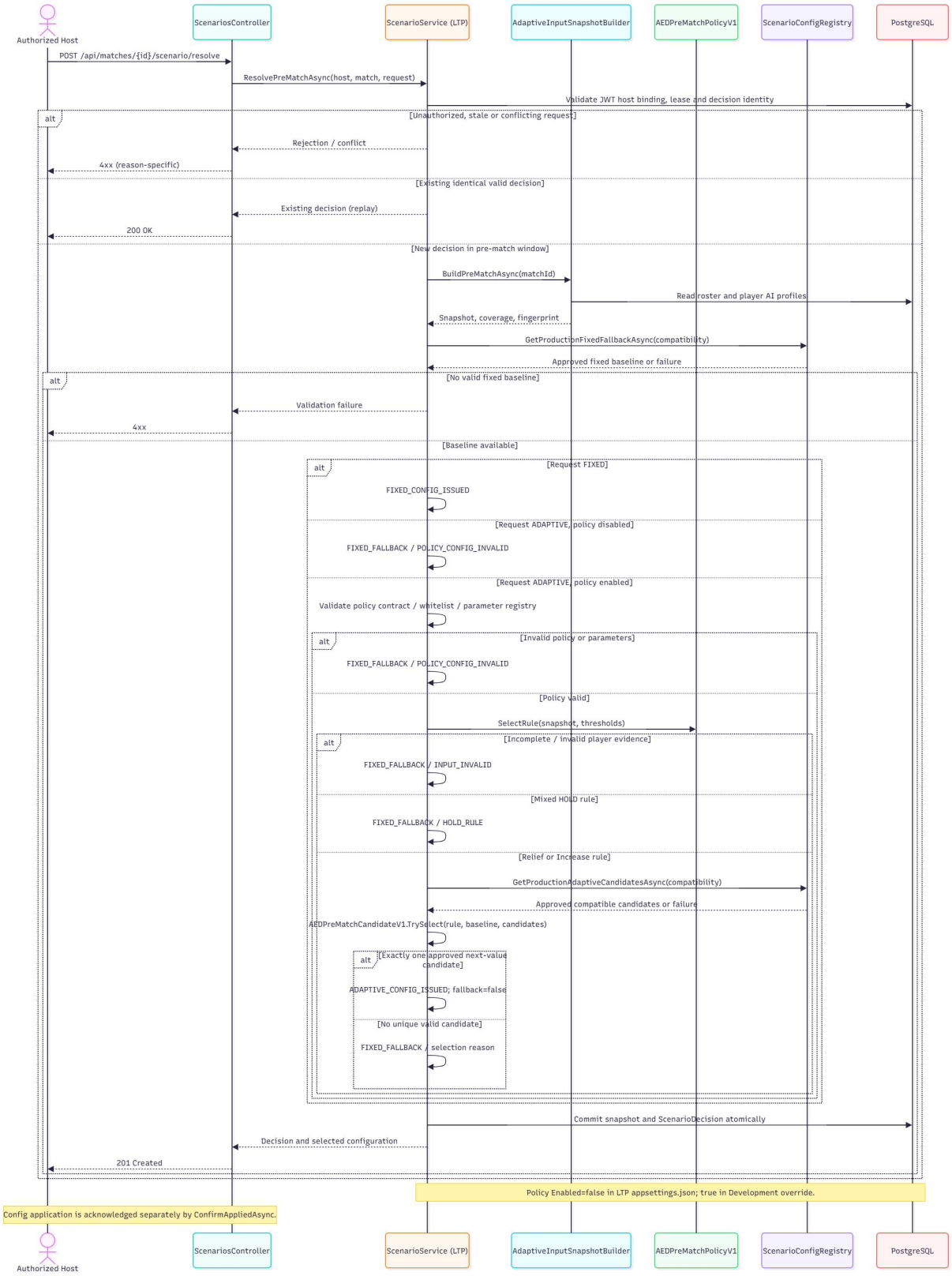


Figure 19. Pre-match scenario resolution with bounded Adaptive selection and Fixed Fallback

ScenarioService loads an approved fixed baseline for every resolution. When Adaptive mode is requested and AEDPreMatchPolicy is enabled, it validates the policy contract and observed Survival/Noise team evidence, then selects an allowed configuration within bounded support-item/chase-speed candidates. Missing evidence, invalid policy or unavailable candidates lead to a fixed fallback with a recorded reason. The decision, fingerprint and applied configuration are persisted for traceability; in-match pilot extensions remain separately gated.

3.7 Match Result and Reward Processing

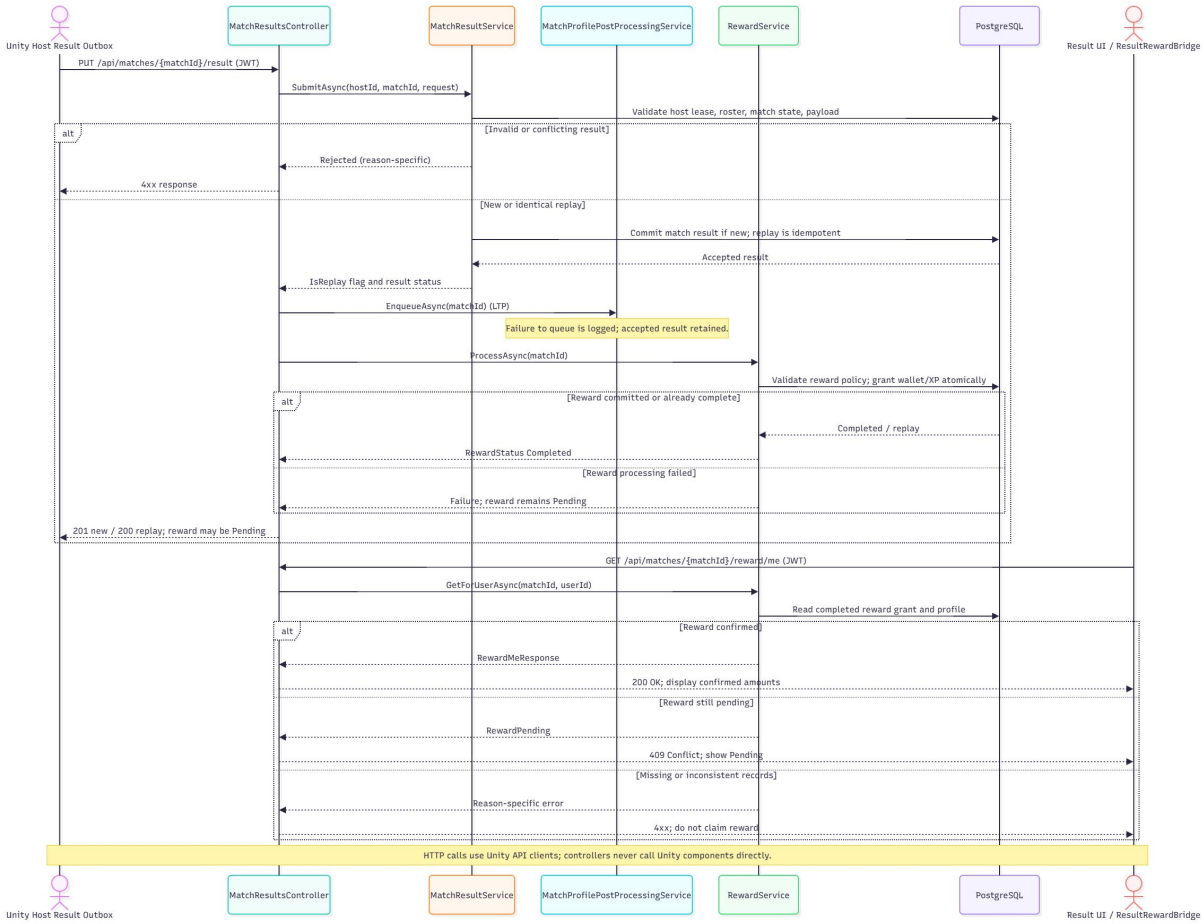


Figure 20. Match result validation, replay handling and reward-processing flow

Class / Component	Design responsibility / selected operations
MatchResultsController	Receives PUT /api/matches/{matchId}/result with authenticated user.
MatchResultService	Checks Host binding, match status, lease, roster, payload and idempotent replay.
RewardService	Applies configured reward/progression policies and stores one set of grants.
ResultRewardBridge	Loads confirmed player rewards through the backend API and distinguishes confirmed, pending and failed reward states.

3.8 Virtual Shop, Wallet and Inventory

Field	Specification
Purpose	Buy eligible items using persistent ECHO Credits, grant ownership and maintain an auditable ledger.
Inputs / triggers	Authenticated UserId, ShopItemId and IdempotencyKey.
Authority / owner	Backend PurchaseService and PostgreSQL transaction; Unity only requests and renders server-accepted state.
Failure / negative path	Inactive/unavailable item, already owned, insufficient balance, conflicting key, concurrency and unauthorized requests never charge twice.

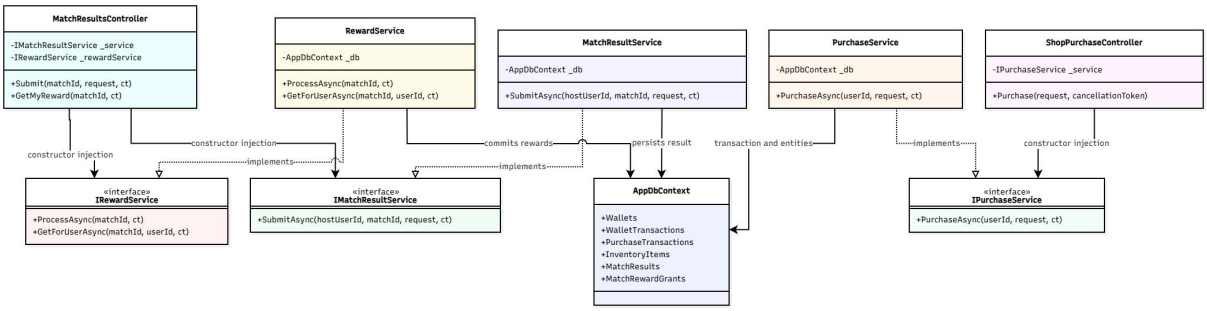


Figure 21. Backend economy and result service dependencies

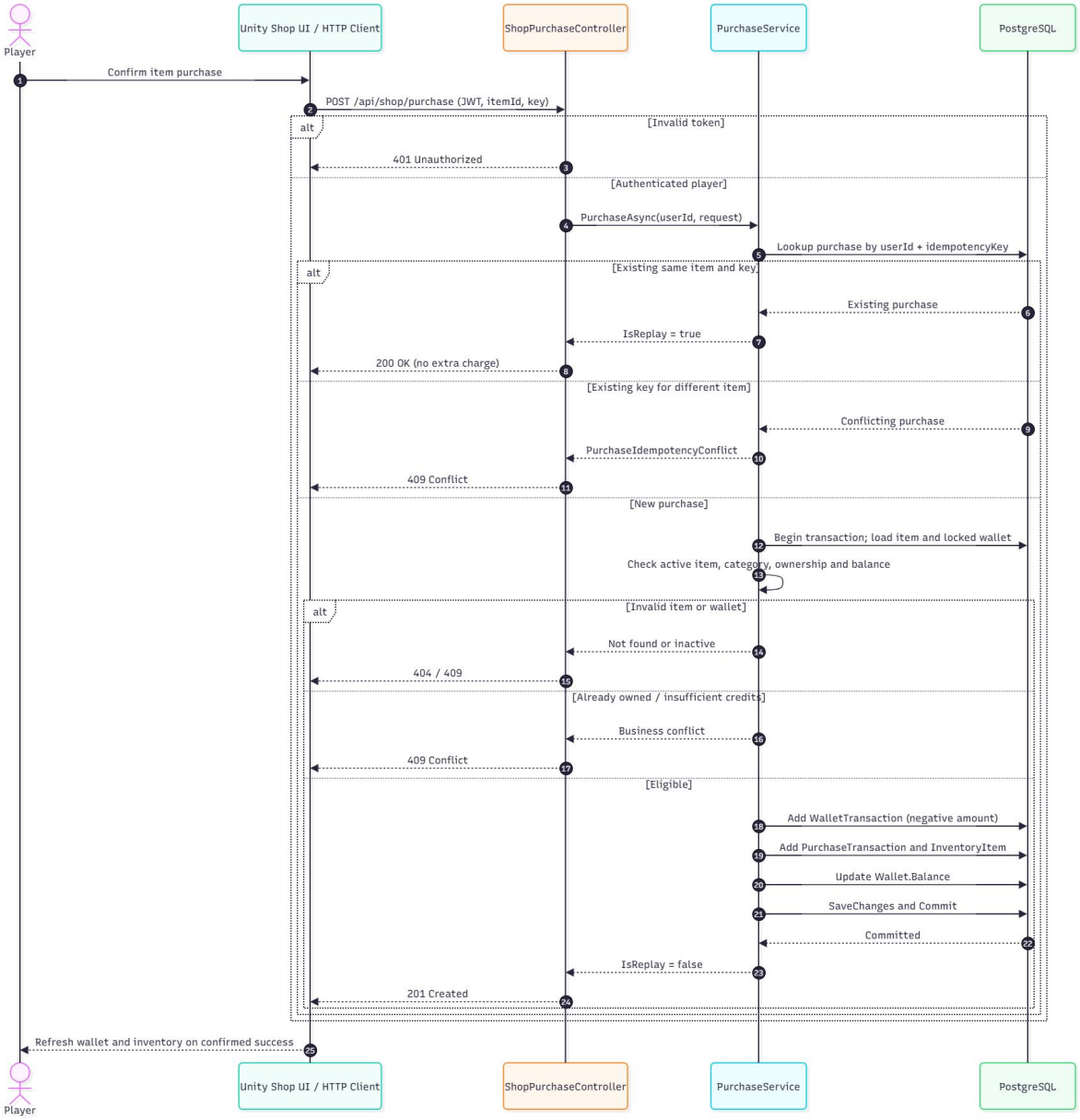


Figure 22. ECHO Credits purchase with atomic updates and replay safety

Class / Component	Design responsibility / selected operations
ShopPurchaseController	Protected POST /api/shop/purchase endpoint.
PurchaseService	Validates price/availability/ownership/balance and handles replay.
Wallet / WalletTransaction	Balance plus immutable debit/grant history.
PurchaseTransaction / InventoryItem	Persistent purchase identity and granted ownership.
InventoryService / LoadoutService	Reads and validates owned/compatible equipment selection.

The registered economy uses ECHO Credits only: validated match rewards or authorized administrative/test allocations. Real-money checkout and payOS are outside the acceptance scope; the limited Admin UI supports permitted, audited operations.

3.9 UI, HUD and Gameplay Feedback

Field	Specification
Purpose	Display mission status, equipment, interaction prompt, teammate life and authoritative result without inventing accepted state.
Inputs / triggers	NetworkMatchState, PlayerLifeState, scene interactions, backend reward/shop responses.
Authority / owner	Local UI renders replicated gameplay state or authenticated API response; server/Host owns accepted mutations.
Failure / negative path	Hide unsupported/inapplicable prompts; show rejected actions, disconnected status, pending reward and session failure correctly.
SRS trace	FR-UI-01–05

Class / Component	Design responsibility / selected operations
Main menu / Lobby UI	Show authorized navigation, selected room/mode/difficulty, ready state and failure feedback.
PlayerInteraction.PromptChanged	Contextual interaction message based on current eligible target.
Match/Player HUD presenters	Read networked objective, status and tool state; update only on actual transitions.
ResultSession / ResultRewardBridge	Distinguish local match outcome from accepted backend reward.
Admin UI	Shows records permitted by backend role; no direct database modification.

Design rule: HUD should be event/state-driven rather than showing all objectives or warnings continuously. Scene-specific art, animation and sound clip lists belong to the GDD/asset inventory; this SDD documents their triggering contracts rather than replicating the full content catalogue.

3.10 Main Business Workflows

Four end-to-end workflows cover authentication and lobby admission, Expedition progression, Stalker encounters and player survival, and result/reward with the virtual economy. All workflows use the selected 1–4-player Expedition scope and Host-authoritative state changes. The diagrams describe design paths; test execution is documented separately in Report 5.

3.10.1 WF-01 – Authentication, Legal Agreement and Lobby

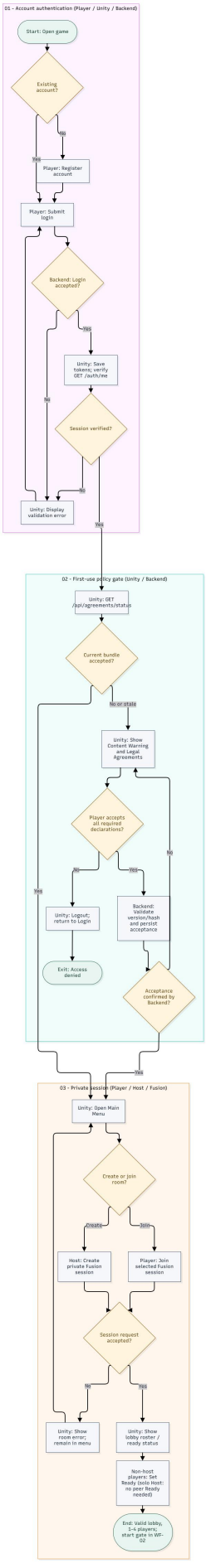

01 - Account authentication (Player / Unity / Backend)
Start: Open game
Existing account?
No
Yes
Player: Register account
Player: Submit login
Backend: Login accepted?
Yes
Unity: Save tokens; verify GET /auth/me
No
Session verified?
No
Unity: Display validation error
Yes
02 - First-use policy gate (Unity / Backend)
Unity: GET /api/agreements/status
Current bundle accepted?
No or stale
Unity: Show Content Warning and Legal Agreements
Player accepts all required declarations?
No
Yes
No
Yes
Unity: Logout; return to Login
Exit: Access denied
Backend: Validate version/hash and persist acceptance
Acceptance confirmed by Backend?
Yes
03 - Private session (Player / Host / Fusion)
Unity: Open Main Menu
Create or join room?
Create
Join
Host: Create private Fusion session
Player: Join selected Fusion session
Session request accepted?
No
Yes
Unity: Show room error; remain in menu
Unity: Show lobby roster / ready status
Non-host players: Set Ready (solo Host: no peer Ready needed)
End: Valid lobby, 1–4 players; start gate in WF-02

Figure 23. Authentication, Legal Agreement and Lobby Workflow

Field	Description
Owner	Hứa Thành Đạt
Actors / Components	Guest, Player, Host Player, Unity Client, Backend API, Photon Fusion
Trigger	A user registers or logs in and requests access to a private session.
Business Outcome	A user with a verified session and current legal acceptance enters a valid 1–4-player lobby; solo Host is supported.
State Changes	Unauthenticated → Authenticated; Agreement Missing/Outdated → Accepted; Not Joined → In Lobby / Ready.
Related FRs	FR-AUT-01–05; FR-MUL-01–03; FR-UI-01.
Related Test Cases	TC-WF01-01 Valid login/lobby; TC-WF01-02 first-use legal acceptance; TC-WF01-03 invalid credentials, declined agreement or room.

3.10.2 WF-02 – Expedition Gameplay and Match Outcome

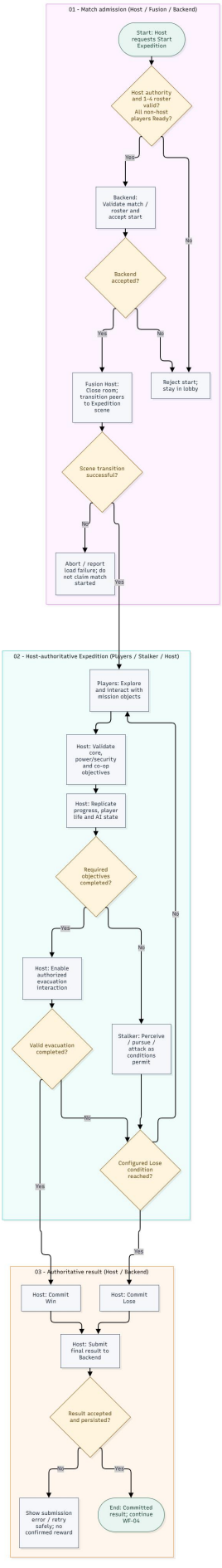

01 - Match admission (Host / Fusion / Backend)
Start: Host requests Start Expedition
Host authority and 1-4 roster valid? All non-host players Ready?
Yes
No
Backend: Validate match / roster and accept start
Backend accepted?
Yes
No
Fusion Host: Close room; transition peers to Expedition scene
Reject start; stay in lobby
Scene transition successful?
No
Yes
Abort / report load failure; do not claim match started
02 - Host-authoritative Expedition (Players / Stalker / Host)
Players: Explore and interact with mission objects
Host: Validate core, power/security and co-op objectives
Host: Replicate progress, player life and AI state
Required objectives completed?
Yes
No
Host: Enable authorized evacuation interaction
Valid evacuation completed?
Stalker: Perceive / pursue / attack as conditions permit
No
Configured Lose condition reached?
No
Yes
Yes
03 - Authoritative result (Host / Backend)
Host: Commit Win
Host: Commit Lose
Host: Submit final result to Backend
Result accepted and persisted?
No
Yes
Show submission error / retry safely; no confirmed reward
End: Committed result; continue WF-04

Figure 24. Expedition Gameplay and Match Outcome Workflow

Field	Description
Owner	Đặng Huy Hoàng
Actors / Components	Host Player, Players, Unity Client, Photon Fusion, Stalker AI, Backend API
Trigger	A Host requests to start Expedition from a valid lobby.
Business Outcome	Host validates match start and objectives, synchronizes Win/Lose, and submits an authorized result for Backend validation.
State Changes	Lobby → In Match → Match Ended; Objective Phases → Escape/Failure; Result Not Committed → Win/Lose Committed.
Related FRs	FR-MUL-03–05; FR-PLY-01–05; FR-EXP-01–05; FR-AI-01–05; FR-UI-02–03.
Related Test Cases	TC-WF02-01 successful Expedition; TC-WF02-02 configured Lose; TC-WF02-03 invalid start, premature evacuation or scene failure.

3.10.3 WF-03 – Stalker AI Gameplay and Player Survival

Figure 25. Stalker AI Gameplay and Player Survival Workflow

Field	Description
Owner	Lâm Thanh Phúc
Actors / Components	Player, Teammate, Host / State Authority, Stalker AI Runtime, Photon Fusion clients
Trigger	An active Expedition player becomes a valid visible target or produces an eligible noise cue.
Business Outcome	Host evaluates Stalker perception, pursuit/search and validated attacks. Player consequences and eligible revive outcomes are replicated to all clients.
State Changes	Stalker: PATROL, DETECT, CHASE, ATTACK, RECOVER, SEARCH; Player: Alive → Caught/Downed → Revived Alive or Eliminated under rules.
Related FRs	FR-AI-01–05; FR-PLY-01–04; FR-MUL-04; FR-UI-02.
Related Figures	Figures 11–12 and 15–17 (life state, revive, Stalker components/FSM/attack).
Related Test Cases	TC-WF03-01 sight/detection; TC-WF03-02 noise/search; TC-WF03-03 authoritative attack; TC-WF03-04 revive; TC-WF03-05 UV does not interrupt Stalker.

Scope note: UV does not stun, repel or interrupt the Expedition Stalker. Adaptive AI remains documented separately in Section 3.6 and Figures 18–19.

3.10.4 WF-04 – Match Result, Reward and Virtual Economy

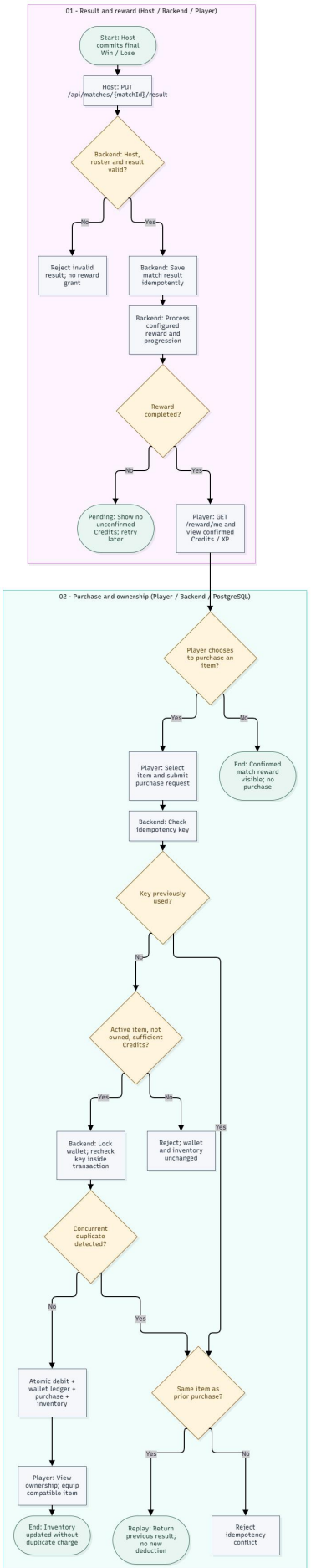

01 - Result and reward (Host / Backend / Player)
Start: Host commits final Win / Lose
Host: PUT /api/matches/{matchId}/result
Backend: Host, roster and result valid?
No
Yes
Reject invalid result; no reward grant
Backend: Save match result idempotently
Backend: Process configured reward and progression
Reward completed?
No
Yes
Pending: Show no unconfirmed Credits; retry later
Player: GET /reward/me and view confirmed Credits / XP
02 - Purchase and ownership (Player / Backend / PostgreSQL)
Player chooses to purchase an item?
Yes
No
Player: Select item and submit purchase request
End: Confirmed match reward visible; no purchase
Backend: Check idempotency key
Key previously used?
No
Yes
Active item, not owned, sufficient Credits?
Yes
No
Backend: Lock wallet; recheck key inside transaction
Reject; wallet and inventory unchanged
Concurrent duplicate detected?
No
Yes
Atomic debit + wallet ledger + purchase + inventory
Same item as prior purchase?
Yes
No
Player: View ownership; equip compatible item
End: Inventory updated without duplicate charge
Replay: Return previous result; no new deduction
Reject idempotency conflict

Figure 26. Match Result, Reward and Virtual Economy Workflow

Field	Description
Owner	Nguyễn Nhật Lệ Thuyên
Actors / Components	Host Player, Player, Unity Client, Backend API, PostgreSQL
Trigger	Host submits a finalized Win/Lose result; a player may subsequently choose to purchase an item.
Business Outcome	Backend validates and persists results/rewards without duplication. Optional valid purchases debit Credits and grant ownership atomically.
State Changes	Result Not Stored → Stored; Reward Pending → Completed on confirmation; Wallet balance updated; unowned item → owned after purchase.
Related FRs	FR-ECO-01–05; FR-UI-03–04.
Related Test Cases	TC-WF04-01 reward and purchase; TC-WF04-02 pending reward/replay; TC-WF04-03 insufficient Credits and duplicate/idempotency conflict.

Verification boundary: TC-WF01–TC-WF04 identifiers are proposed Report 5 references, not claims of executed or passed tests. Adaptive Candidate selection is not a prerequisite for the Stalker encounter workflow.